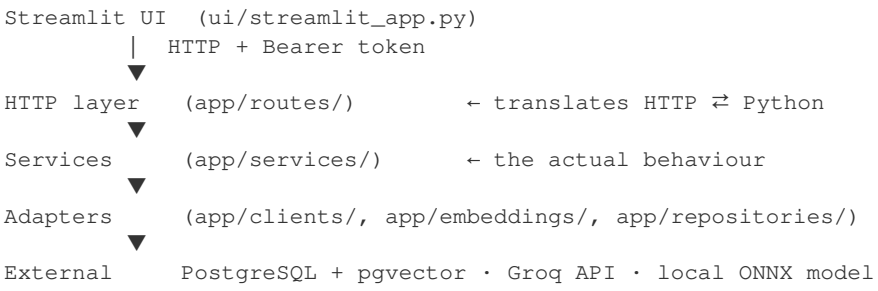
OmniAssist — File Guide

Every file in the codebase: what it is, why it exists, and where it sits in the system. Written for revision and for interviews, where the question is rarely *"what does this do"* and almost always *"why is it there, and what breaks without it"*.

Companion documents: **`LEARN_SHEET.md`** (concepts C1–C75, decisions D1–D30) and **`BUILD_LOG.md`** (the step-by-step record of how it was built).

The shape of the system



The dependency rule: every arrow points downward, and each layer depends on the *interface* of the layer below it, never on a concrete class. That single constraint is what makes the test suite run with no API key and no database — a fake satisfying the same interface is substituted at the boundary.

`app/domain.py` sits outside the stack: everything may import it, it imports nothing.

Entry point and configuration

`app/main.py`

Is — the FastAPI application: creates `app`, mounts the auth and chat routers, exposes `GET /healthz`.

Why — something has to assemble the application, and it is also where **fail-fast configuration** is re-established. `get_settings()` is called inside the `lifespan` handler, so a missing or malformed setting stops the process from ever accepting traffic instead of failing on some user's first request.

Fits — the composition root for HTTP. Imported by `uvicorn` and by nothing else in the app.

`app/config.py`

Is — the `Settings` model (database URL, JWT secret, LLM provider and model, chunk size and overlap, embedding provider) plus `get_settings()`, cached with `lru_cache`.

Why — one typed, validated place for every value that differs between machines. Secrets use `SecretStr` so they cannot be printed in a log or traceback. Settings load **lazily**: importing a module must have no side effects, or tests and CI could not import the app without credentials.

Fits — read by the factories and by `main.py` at startup. Nothing else should read environment variables directly.

`.env.example`

Is — the template listing every variable the app needs, with no real values.

Why — the real `.env` is git-ignored, so this is the only record of what a new machine must provide.

`app/__init__.py` and the other empty `__init__.py` files

Is — package markers.

Why — they make `app`, `app.clients`, `app.routes` and the rest importable packages, which is what lets `from app.services.chat import ChatService` work from anywhere, including tests.

Domain

`app/domain.py`

Is — the shared vocabulary: `Message`, `Vector`, the frozen `User` and `Chunk` dataclasses.

Why — these types cross every layer. If they lived in the database module, the service layer would import the database to describe a message; if they lived in the HTTP module, the repository would import FastAPI. A dependency-free module in the middle prevents both.

Fits — the one module with a hard rule written at the top of the file: *it must not import from any other application module.* Everything else may import it.

Adapters — the language model

`app/clients/base.py`

Is — the `LLMClient` **Protocol** (`chat`, `stream_chat`) and `LLMError`.

Why — the application depends on this declaration rather than on a vendor SDK, so changing provider is a configuration change. `Protocol` rather than an abstract base class means the test fake satisfies it **without inheriting from anything** — structural typing, checked by mypy.

Fits — the contract between `services/chat.py` and whichever provider is configured.

`app/clients/groq_client.py`

Is — the Groq implementation of that Protocol.

Why — Groq's free tier (~1,000 requests/day, no card) is what makes a zero-budget project possible. This is also the **error boundary**: whatever the vendor raises is translated into `LLMError`, so no caller ever handles a Groq exception type.

Fits — one of potentially several implementations; the only file that knows Groq exists.

`app/clients/factory.py`

Is — `get_llm_client()`, cached, returning the configured implementation.

Why — the composition root for the LLM. Callers ask for *an* `LLMClient` and never name a concrete class, which is what keeps the dependency rule intact.

Adapters — embeddings

`app/embeddings/base.py`

Is — the `Embedder` Protocol (`model_id`, `dimensions`, `embed_documents`, `embed_query`), `EmbeddingError`, and `EMBEDDING_DIMENSIONS = 384`.

Why — embeddings are *not* interchangeable the way LLM providers are. A vector is only meaningful against the model that produced it: dimensions differ between models, and even at equal width the

semantic spaces differ, so a mismatch returns plausible nonsense rather than an error. Hence `dimensions` (so the schema is derived, not guessed) and `model_id` (so stale vectors can be *detected* after a provider change).

The width constant lives here, not in `local.py`, because `db/models.py` needs it — and importing `local.py` would drag `onnxruntime` into every database import.

Fits — the contract between the retrieval pipeline and whatever produces vectors.

app/embeddings/local.py

Is — `LocalEmbedder`: `bge-small-en-v1.5` via `fastembed`, running on this machine.

Why — hosted embedding APIs cost money per request; this one costs nothing and needs no key, which also keeps CI free of network calls. Queries get an instruction prefix that documents do not, because that is what the model was trained to expect.

Fits — the default implementation. Downloads ~130 MB on first construction, which is why nothing constructs it at import time and no test touches it.

app/embeddings/factory.py

Is — `get_embedder()`, cached, switching on `embedding_provider`.

Why — same composition-root role as the LLM factory, with one extra trick: the import of `LocalEmbedder` sits **inside the function**, so importing the factory does not load `fastembed` or `onnxruntime`. `"gemini"` is already a valid config value and raises `NotImplementedError` — the axis of change is documented in the type, and the gap fails loudly.

Adapters — persistence

app/db/models.py

Is — the SQLAlchemy tables: `Conversation`, `MessageRow`, `UserRow`, `DocumentChunk`.

Why — one declarative description of the schema, which Alembic diffs against the live database to generate migrations. `DocumentChunk` carries the `vector(384)` column, plus `document_name` and `chunk_index` so an answer can cite where it came from, and `model_id` so vectors from a different embedder are identifiable.

Fits — below the repositories. Nothing above the repository layer imports this module; services deal in `app/domain.py` types.

app/db/session.py

Is — `get_engine()` and `get_session_factory()`, both cached.

Why — an engine owns a connection pool and must exist once per process, not once per request. `pool_pre_ping=True` checks a connection before handing it out, so a database restart doesn't surface as a random error.

app/repositories/base.py

Is — the `ConversationRepository`, `UserRepository` and `ChunkRepository` Protocols, plus `EmailAlreadyExistsError` and `ConversationNotFoundError`.

Why — storage behind an interface, for the same reason as the LLM. The errors are deliberately vendor-neutral: a caller must never catch `sqlalchemy.exc.IntegrityError`, or the storage engine

has leaked upward. One error covers both *"does not exist"* and *"is not yours"* — telling a caller which would let them enumerate real IDs.

`app/repositories/postgres.py`

Is — the real implementations against PostgreSQL.

Why — this is where **ownership is enforced**, inside the `WHERE` clause rather than as a check beside the query. There is no way to read a conversation or search chunks without also proving the caller owns them. A service-level check can be forgotten by the next code path; a filter in the query cannot be, because there is no query without it.

Chunk search orders by **cosine distance** (`<=>`), matching what the embedding model was trained for — a different operator would silently rank worse.

`app/repositories/memory.py`

Is — in-memory implementations of the same three Protocols, plus a hand-written `cosine_distance()`.

Why — the test double. The suite runs with no database because these satisfy the same interfaces. The distance function is written out rather than imported so that what pgvector's `<=>` actually computes is visible and testable.

Fits — used only by tests, but lives in `app/` because it implements an application interface and must stay in step with it.

Services — the behaviour

`app/services/chat.py`

Is — `ChatService`: loads history, calls the model, streams the reply, persists both sides of the exchange.

Why — the orchestration that is neither HTTP nor storage. It also carries the project's most instructive bug: written originally as a generator, **none of its body ran until the first chunk was requested**, so the ownership check fired *after* the response headers had gone out and could no longer produce a correct status code. The fix was an eager/lazy split — validation runs immediately, only streaming is deferred.

It also owns **grounding**. When a retriever is attached, retrieved passages are prepended as a system message for that one model call — and the **raw question** is what goes into history. Storing the augmented prompt would grow history by thousands of characters per turn and re-send context retrieved for an older question. With nothing retrieved the prompt is left alone, since an empty context block makes the assistant refuse questions it could answer.

Fits — called by `routes/chat.py`, depends on the repository, LLM and retriever interfaces, knows nothing about FastAPI.

`app/services/indexing.py`

Is — `Indexer`: turns an uploaded PDF into stored, embedded passages.

Why — the write-side mirror of `Retriever`, composing extraction, chunking and embedding in order. Two properties matter more than the composition: re-uploading a document **replaces** its passages rather than appending (otherwise retrieval spends half its context budget repeating itself),

and extraction and embedding both complete **before** anything is deleted or written, so a failed upload can neither leave a half-indexed document behind nor destroy a good earlier version.

Fits — called by `routes/documents.py`. Holds an `Embedder` and a `ChunkRepository` as interfaces, so tests index a real PDF with no model and no database.

app/services/retrieval.py

Is — `Retriever`: embeds a question and returns the passages closest to it.

Why — its own class rather than two more collaborators bolted onto `ChatService`, which already has a client and a repository. This owns one job — *turn a question into relevant passages* — and can be tested without a conversation, a client or a history. A blank question retrieves nothing, because embedding whitespace returns an arbitrary nearest neighbour and would present it to the model as relevant.

Fits — between `ChatService` and the chunk repository. Holds an `Embedder` and a `ChunkRepository`, both as interfaces, so tests use a fake embedder and a dictionary.

app/services/pdf.py

Is — `extract_text(data: bytes) -> str` and `PdfExtractionError`.

Why — stage one of retrieval. Takes bytes rather than a path or an upload object, which keeps it free of filesystem and HTTP concerns and testable in one line. One `except PyPdfError` contains all fourteen of the library's error types, because they share a base class. A document with **no text layer** — a scan — is rejected with an OCR hint rather than indexed silently: an upload that appears to succeed while storing nothing is a worse failure than an error.

app/services/chunking.py

Is — `chunk_text(text, *, size, overlap) -> list[str]`.

Why — stage two, and the biggest quality knob in retrieval. A whole document embeds to the average of everything it says, which matches nothing; the unit you store is the unit you get back. Overlap ensures an idea cut across a boundary still appears whole somewhere.

Keyword-only arguments because `chunk_text(body, 150, 1000)` type-checks perfectly and silently produces garbage. `start` is derived from where the chunk *actually* ended, never from a fixed stride — the version that did the latter dropped 32 of 45 characters with no error at all.

HTTP layer

app/routes/chat.py

Is — the chat endpoint, streaming replies over Server-Sent Events.

Why — SSE rather than WebSockets because the traffic is one-directional and SSE is plain HTTP: no special proxy handling, automatic reconnection. Translates service exceptions into status codes — notably **404, not 403**, for another user's conversation.

app/routes/auth.py

Is — `POST /auth/signup`, `POST /auth/login`, `GET /auth/me`.

Why — the credential boundary. Unknown email and wrong password return **identical** errors, and login compares against a dummy hash when the email is unknown so response time does not reveal

whether an account exists.

app/routes/documents.py

Is — `POST /documents` (upload and index) and `GET /documents` (list names).

Why — the upload boundary. Size is checked **before** the body is read, because `await file.read()` on a huge upload exhausts memory and FastAPI imposes no limit of its own; the length is re-checked afterwards, since a declared size is a client claim. Errors map deliberately: **400** for a bad PDF, passing the extraction message through because it was written for the user and says what to do; **413** for oversize; **503** for an embedding failure, which is our fault and says nothing about our internals.

Fits — above `Indexer`, below HTTP. Indexing happens inside the request for now; moving it to a queue is v0.3 work (D36).

app/routes/deps.py

Is — `get_current_user`, the `CurrentUser` alias, and `get_user_repository`.

Why — one place that turns `Authorization: Bearer <token>` into a `User`, so every protected route gets identical behaviour. The user is **looked up**, not trusted from the token alone: a JWT cannot be revoked before it expires, so a deleted account would otherwise keep access. `HTTPBearer(auto_error=False)` is deliberate — FastAPI's own default returns 403 for a missing header, which is the wrong code.

app/schemas/auth.py

Is — the Pydantic request and response models.

Why — validation at the edge, once. Email normalisation happens here so no downstream code has to think about case. A response model is a **filter**: it determines what leaves the system, which is what stops a password hash being serialised by accident.

Security

app/security/password.py

Is — Argon2id hashing and verification.

Why — Argon2id won the Password Hashing Competition and is deliberately *memory*-hard, so attacking it in parallel on GPUs gains far less than against a fast hash. Hashing is not encryption: there is no way back, which is the point.

app/security/tokens.py

Is — HS256 JWT creation and decoding.

Why — stateless sessions: the server verifies a signature instead of looking up a session. The verification algorithm is **pinned**, which closes the algorithm-confusion attack where a forged token declares its own algorithm (including `none`). A JWT is signed, not encrypted — its contents are readable by anyone holding it, so nothing secret goes inside.

User interface

ui/styles.py

Is — the interface's CSS, plus a `brand()` helper, as a single string.

Why — Streamlit's theming API covers only a few colours, so everything past that is CSS aimed at its `data-testid` attributes. Keeping it here leaves the app module about behaviour. Those attributes are implementation details rather than a contract, so the selectors are deliberately narrow: an early blanket `[class*="st-"]` font rule also matched Streamlit's icon spans and rendered the password-reveal glyph as the literal text "visibili".

Fits — imported by `streamlit_app.py` and injected once per run. Paired with `.streamlit/config.toml`, which sets the theme tokens Streamlit applies to its own chrome before any of this CSS runs.

`.streamlit/config.toml`

Is — theme colours, minimal toolbar, and `maxUploadSize`.

Why — the tokens here style what Streamlit draws itself, which CSS cannot reach. `maxUploadSize = 10` matches `max_upload_bytes` in `app/config.py`, so the dropzone advertises the real limit instead of the 200 MB default.

`ui/streamlit_app.py`

Is — the web interface: login and signup tabs, chat with streaming output, a sidebar PDF uploader with the list of indexed documents, and logout.

Why — a usable front end in Python while the API stabilises; React arrives in v0.4. The UI calls the API over HTTP rather than importing the services, so the API key never reaches the browser and the backend stays the only place that talks to the model.

Named `streamlit_app.py` rather than `app.py` because `ui/app.py` collided with the `app/` package.

Database migrations

`alembic.ini` and `migrations/env.py`

Is — Alembic's configuration and runtime wiring; `env.py` points Alembic at `Base.metadata` and at the database URL from `Settings`.

Why — so migrations use the same configuration as the application rather than a second copy of the connection string.

`migrations/versions/7f0eab0265ea_create_conversations_and_messages.py`

First schema: conversations and messages.

`migrations/versions/40ea924b41d1_create_users.py`

Adds the users table.

`migrations/versions/b05d08fb5171_add_user_id_to_conversations.py`

Adds `conversations.user_id` — the column that makes ownership possible. Autogenerate emitted `nullable=False` against a populated table and PostgreSQL refused, because autogenerate diffs *models* and knows nothing about *data*. In production this is three deploys: add nullable, backfill, then set not-null.

`migrations/versions/c3a71f9e4d02_create_document_chunks.py`

Creates `document_chunks` with the `vector(384)` column. **Hand-written**, because autogenerate would never emit `CREATE EXTENSION IF NOT EXISTS vector` — it has no concept of a missing

extension, and without that line the type does not exist.

The width is frozen as a literal on purpose: a migration must produce the same schema a year from now, so it cannot read today's configuration. No vector index yet — HNSW and IVFFlat are both approximate, a sequential scan at this scale is exact and fast, and IVFFlat built on an empty table locks in poor recall.

Why migrations at all — the schema gets a version history exactly like the code, every change is reversible, and the same sequence runs on every machine.

Tests

`tests/conftest.py`

Is — shared pytest fixtures, including `user_id` and `other_user_id`.

Why — the two-user fixtures exist so cross-user isolation is cheap to assert. Most ownership bugs only appear when a second user exists.

`tests/fakes.py`

Is — `FakeLLMClient`, `FakeEmbedder`, and `make_pdf()`.

Why — the substitutes that make the suite hermetic. `FakeEmbedder` derives deterministic vectors from a hash, so identical text always embeds identically without a model. `make_pdf()` assembles a real PDF byte by byte — objects, xref table, trailer — rather than adding a whole PDF-writing library as a test dependency.

`tests/test_chat_service.py`

Streaming, history accumulation, trimming, defensive copies, and **cross-user isolation** including the regression for the generator bug.

`tests/test_pdf.py`

Extraction, page separation, rejection of non-PDFs, empty files and text-less scans, and a check that **no `pypdf` exception escapes**.

`tests/test_chunking.py`

Sizes, overlap, boundary preference, the `ValueError` guards, termination under a pathological overlap, and the **conservation check** — *no input text is lost* — which is the test that caught the data-loss bug.

`tests/test_embeddings.py`

Determinism, width, empty input, ordering, and a one-line **protocol conformance proof** that mypy checks at build time.

`tests/test_indexing.py`

End-to-end indexing with fakes, **replace-not-append** on re-upload, that a failed extraction stores nothing, that a failed re-upload leaves the previous version intact, and per-user isolation of both upload and delete.

`tests/test_retrieval.py`

Closest-passage ordering, `top_k`, per-user scoping, blank questions, an empty store, and that citation details survive retrieval.

`tests/test_chunk_repository.py`
Round-trip, ordering by similarity, `limit`, the mismatched-lengths guard, and **another user's chunks being invisible**.

Project and CI

`pyproject.toml`
Is — project metadata, runtime dependencies, the `dev` dependency group, and tool configuration for pytest and mypy.

Why — one file instead of `setup.py`, `requirements.txt` and friends. Dev tools are a separate group so production installs do not pull in pytest and mypy.

`uv.lock`
Is — the exact resolved version of every package, transitive ones included.

Why — reproducibility. `pyproject.toml` says *"FastAPI 0.141 or newer"*; the lockfile says *exactly which build ran*. CI uses `uv sync --locked` so it installs what was tested, not whatever was newest that morning.

`.github/workflows/ci.yml`
Is — the quality gate: format check, lint, type check, tests, on every push and pull request.

Why — review should be about design, not spacing. The hard constraint is that **CI never has an API key or a database**, which is what forced every external dependency behind an interface from the beginning. A pipeline that needs secrets to run is a pipeline that stops running.

`README.md`
What the project is and how to run it.

`BUILD_LOG.md`
The chronological record: what was built in each step, in order.

`DESIGN_v0.3.md`
The design for the tool-use framework, written before any code. Settles how tools are declared, how the loop is bounded, why streaming and tools conflict, what happens when a tool fails, why `Message` must stop being a `dict`, and why Block 1 ships no tool with side effects.

`WHY.md`
The reasoning behind every decision from v0.1 onward: what the problem was, what was built, **what was rejected and why**, what the architecture gained, and what it cost. Includes a chapter on the four defects that shipped and what each taught, and the questions an interviewer is likely to ask.

`LEARN_SHEET.md`
Concepts C1–C75 and decisions D1–D30, each with the alternative that was rejected and why. The primary interview source.

`FILE_GUIDE.md`
This document.

Updated through v0.2 (shipped) and the v0.3 design. Revise when files are added, removed, or change purpose.